ECM2433 Things To Know Before The Exam

Fast review pack generated from the RAG tracker, revision spreadsheet, and mock-exam mistakes. Use before the exam; do not take it into a closed-book exam unless explicitly permitted.

How to use this in the last 30 minutes

- First 10 minutes: read the personal priority table and the pinned traps.
- Next 15 minutes: scan the C, C++, and Rust rule pages, especially red/amber topics.
- Final 5 minutes: test yourself from the mental checklist, not by rereading passively.

The counts below are weighted from open mock-exam mistakes, open spreadsheet rows, and red RAG topics. High count does not mean ignore the rest; it means check these first.

Priority area	Open mock	Open sheet	R topics	Why it matters
C pointers, memory, arrays, strings	1	1	15	High bug density: bounds, allocation size, terminators, UB.
Rust ownership, borrowing, strings, slices	4	0	6	Core Rust marks depend on exact ownership/borrow wording.
C preprocessor, headers, build	2	2	7	Small syntax details decide many marks: tabs, guards, macro grouping.
C files, streams, buffering, diagnostics	1	2	5	Exam questions often hide return checks and stream-state details.
Rust Result, Option, parsing, tests	3	0	4	Most Rust error-handling answers need exact `Result`/`?` behaviour.
C++ templates, operators, STL algorithms	1	1	5	Type signatures and return/reference choices are mark-heavy.
Rust iterators, traits, closures, concurrency	2	0	3	Iterator item/reference types and closure capture are common traps.
C++ classes, inheritance, polymorphism	2	0	2	Access and virtual dispatch questions are easy to misread.

Pinned Personal Traps

- Macro answers need full parentheses and multi-statement macros should use do { ... } while (0).
- For C strings, always account for the final '\0'; `sizeof` and `strlen` answer different questions.
- After `malloc`, check for failure before writing. After `free`, the pointer value is stale; do not dereference it.
- For file/stream questions, say what is returned and what must be checked: `scanf`, `fgets`, `fread`, `ferror`, `errno`.
- C++ public inheritance means is-a. `protected` is visible to derived classes; `private` is not directly accessible.
- Polymorphic base classes need virtual functions for dispatch and a virtual destructor for deletion through base pointer type.
- Rust `&str` is a borrowed string slice; `String` owns heap text. Prefer `&str` for read-only text parameters.
- Rust `?` unwraps `Ok` and returns `Err` early. It is explicit propagation, not a hidden exception.
- Iterator item types matter: `.iter()` gives references; `.into_iter()` on a `Vec<T>` gives owned `T` values, but `filter` borrows its item.

Inputs used: 78 RAG topics, 80 mock-exam mistake entries, and 141 spreadsheet mistake rows.

C: Must-Know Rules

Types, conversions, and arrays

- `7 / 2` with integers gives `3`; cast before division for floating result: `(double)a / b`.
- Unsigned arithmetic wraps modulo the type range. `unsigned u = 0; u - 1` gives the maximum unsigned value.
- `char` values are commonly promoted to `int` for arithmetic, often using the character code.
- `sizeof array` works only while the real array type is visible. Function parameters like `int values[]` are pointers.
- C arrays are row-major: for `int A[rows][cols]`, offset of `A[r][c]` is `r * cols + c` integers.

Pointers, allocation, and strings

- `int *p` creates a pointer variable. It does not allocate an `int`; `p` must point at valid storage before `*p = ...`.
- Use `malloc(n * sizeof *p)` or `malloc(n * sizeof(int))`; `sizeof *p` follows the pointed-to type if it changes.
- Check `p != NULL` before writing through allocated memory. Free each allocation exactly once.
- A wild pointer is uninitialised. A dangling pointer used to point at valid memory, but that memory is no longer valid.
- `strcpy` and `strcat` receive pointers. Destination must point to writable space large enough for all characters and `\0`.
- `sizeof s` measures storage bytes for an array; `strlen(s)` counts visible characters before `\0`.

```
char str[] = "By Divine Aid";
char *dup, *p, *q;

for (p = str; *p; p++)
    ;
dup = malloc((size_t)(p - str + 1) * sizeof *dup);
assert(dup != NULL);
for (p = str, q = dup; *p; )
    *q++ = *p++;
*q = '\0';
```

Files, streams, and buffering

- `FILE *fp` is a stream handle, not the file itself. Always check `fp == NULL` after `fopen`.
- `scanf("%d", &k)` parses input according to `%d` and writes into `k`, so it needs the address.
- `fgets(buf, size, fp)` reads a line or up to `size - 1` chars and writes a terminator.
- `fread(out, sizeof *out, 1, fp) == 1` means exactly one object was read successfully.
- After a read loop ends on `EOF`, call `ferror(fp)` to distinguish normal end-of-file from a real read error.
- `printf` writes to `stdout`, a buffered stream. To force visible output now, call `fflush(stdout)` after `printf`.
- `stderr` is for diagnostics/errors so normal output and error output can be redirected separately.

Preprocessor, headers, and build

- The preprocessor rewrites source before compilation: includes are copied in, macros are text-substituted.
- Include guards prevent one header's declarations from being processed twice in the same compilation unit.
- `static` on a file-scope helper makes it private to that `.c` file. `extern` declares something defined elsewhere.
- Macro pitfalls: precedence, repeated evaluation, and multi-statement expansion. Fully parenthesise macro arguments and result.
- Makefiles: first target is default, prerequisites decide rebuilds, recipes must start with a tab.
- Compile `.c` to `.o`; link `.o` files into an executable. Libraries are collections of object files.

```
#define SQUARE(X) ((X) * (X))     /* still bad for SQUARE(i++) */

#define swap(a, b) do { \
    int tmp = (a);     \
    (a) = (b);         \
    (b) = tmp;         \
} while (0)
```

C++: Must-Know Rules

Classes, access, and inheritance

- `class` members are private by default; `struct` members are public by default.
- A constructor initialiser list directly initialises members and base classes: `MyInteger(int value) : i(value) {}`.
- `private` base members are not directly accessible in derived code. `protected` members are accessible inside derived member functions.
- Public inheritance means is-a. Protected/private inheritance hide the base interface from outside users.
- Use getters/setters or `friend` only when controlled access is needed; making fields public weakens encapsulation.
- Base constructors run before derived constructors. Destructors run in reverse order.

Virtual dispatch and ownership

- Without `virtual`, a call through `Sensor *` or `Sensor &` uses the base type's function.
- `virtual double read() const = 0;` makes `Sensor` abstract and forces derived classes to implement `read`.
- `override` checks the derived signature really matches the base virtual function, including `const`.
- Deleting through a base pointer type requires a virtual destructor: `virtual ~Sensor() = default;`.
- Pass `const Sensor&` when you only need to read an object owned elsewhere. Use `std::unique_ptr<Sensor>` when the pointer owns it.
- Object slicing happens when a derived object is copied into a base object by value; use references or pointers for polymorphism.

```
class Sensor {
public:
    virtual ~Sensor() = default;
    virtual double read() const = 0;
};

std::vector<std::unique_ptr<Sensor>> sensors;
sensors.push_back(std::make_unique<TemperatureSensor>(7, 21.5));
```

RAII, smart pointers, exceptions

- RAII = Resource Acquisition Is Initialization: acquire in constructor/object creation, release in destructor.
- `std::unique_ptr<T>` has exclusive ownership and cannot be copied; transfer with `std::move`.
- `std::shared_ptr<T>` is copyable shared ownership; object is destroyed when the last shared owner is gone.
- `std::weak_ptr<T>` observes a shared object without keeping it alive; call `.lock()` to try getting a `shared_ptr`.
- C++ exceptions propagate up the call stack until caught. Catch standard exceptions as `const std::exception&` and use `.what()`.
- Disadvantage of exceptions: control flow can jump to catch blocks, so paths are harder to follow.

Operators, templates, STL

- `operator[]` should return `T&` if `arr[i] = value` must modify the stored element.
- `operator<<` usually returns `std::ostream&` so output can chain: `std::cout << a << b`.
- `operator+` normally returns a new value and should often be `const`; use `+=` for mutation.
- Templates are patterns. `Stack` is the template name; `Stack<T>` or `Stack<int>` is a type.
- Template definitions usually live in headers because the compiler needs the full definition when instantiating.
- Use `std::map::find(key)` for keyed lookup. Ordered map lookup is logarithmic; vector search is usually linear.
- Algorithms take iterator ranges and predicates/comparators: `sort`, `binary_search`, `count_if`.

```
template<typename T>
T find_max(T a, T b) {
    return a < b ? b : a;
}

auto it = students.find(1001);
if (it != students.end()) {
    std::cout << it->second.name << "\n";
}
```

Rust: Must-Know Rules

Ownership, borrowing, and text

- Ownership rules: each value has an owner; only one owner at a time; value is dropped when owner goes out of scope.
- `String` owns heap text. `&str` is a borrowed string slice and is best for read-only text parameters.
- Passing `String` by value moves ownership. Passing `&String` or `&str` borrows so the caller can still use it.
- You can have many immutable references or one mutable reference, but not both active for the same value.
- A `Vec` push may reallocate, so Rust rejects holding a reference into a `Vec` and then mutating the `Vec` before using the reference.
- `&[T]` is a borrowed slice of contiguous elements. It can accept a whole vector, array, or sub-slice.

```
fn label_len(label: &str) -> usize {
    label.len()
}

let name = String::from("sensor");
let n = label_len(&name);
println!("{name} {n}");
```

Option, Result, and file parsing

- `Option<T>` is `Some(value)` or `None` for possibly missing values.
- `Result<T, E>` is `Ok(value)` or `Err(error)` for recoverable errors.
- `?` unwraps `Ok(value)` and returns `Err(error)` early from the current function.
- `unwrap()` panics with a generic message. `expect("message")` panics with your custom explanation.
- `parse::<i32>()` tells Rust the target parse type. Handle the result with `match`, `?`, or `expect` only when failure is unrecoverable.
- `map_err` converts one error type into another. `ok_or_else` converts `Option` into `Result`.

```
fn parse_port(line: &str) -> Result<u16, String> {
    let (key, value) = line
        .split_once('=')
        .ok_or_else(|| "missing equals sign".to_string())?;
    if key.trim() != "port" {
        return Err("expected port key".to_string());
    }
    value.trim().parse::<u16>().map_err(|_| "bad port".to_string())
}
```

Iterators, traits, closures

- `.iter()` gives references. `.into_iter()` on `Vec<T>` consumes the vector and yields owned `T` values.
- `filter` receives a reference to each iterator item. If the item is `i32`, the filter closure sees `&i32`.
- `enumerate()` gives `(index, value)`. Use brackets around tuple patterns: `|(index, reading)|`.
- `filter_map` filters and maps in one step by returning `Some(output)` or `None`.
- `fold(start, |acc, item| next_acc)` must return the next accumulator value; `acc += ...` returns `()`.
- Trait bounds say what operations generic code may use: `T: PartialOrd + Debug`.
- `#[derive(Debug, PartialEq, Eq, Hash)]` asks Rust to generate common trait implementations.
- A closure that mutates captured state uses `FnMut` and must be bound as `mut` if called more than once.

```
let positives: Vec<(usize, i32)> = readings
    .into_iter()
    .enumerate()
    .filter(|(_, reading)| match reading {
        Some(value) => *value > 0,
        None => false,
    })
    .map(|(index, reading)| {
        let value = reading.unwrap();
        (index, value)
    })
    .collect();
```

Projects, testing, concurrency

- `src/lib.rs` holds reusable library logic. `src/main.rs` should collect input, call library functions, and print output.
- Items are private unless marked `pub`. Integration tests in `tests/` use the crate like outside code, so they need public API.

- Unit tests beside code can use `use super::*;` to access private items in the parent module.
- `Arc<T>` = atomically reference counted shared ownership across threads. `Rc<T>` is not thread-safe.
- `Mutex<T>::lock()` returns a `Result`; the guard unlocks when dropped. Poisoning reports a panic while locked.
- For `mpsc`, clone senders for workers, move each clone into the thread, and `drop(tx)` so receiver iteration can finish.

Cross-Language Comparisons

Issue	C	C++	Rust
Heap ownership	`malloc`/`free`; programmer owns cleanup	RAII objects and smart pointers clean up in destructors	Owner drops value automatically; borrow checker prevents invalid use
Error handling	Return codes/NULL; caller must check	Exceptions propagate to catch blocks	`Result<T, E>` makes errors explicit; `?` propagates
Strings	char arrays/pointers with `\0` terminator	`std::string`, `const std::string&`, `std::string_view`	`String` owns text; `&str` borrows text
Generic code	`void *`, function pointers, macros	Templates, STL iterators, function objects/lambdas	Generics with trait bounds and iterator adapters
Polymorphism	Function pointers/callbacks	Virtual functions through base refs/pointers	Traits and trait objects/generic bounds
Build flow	preprocess, compile to `.o`, link; Makefiles	compile/link with `g++`; CMake for projects	`cargo check`, `cargo build`, `cargo run`, `cargo test`

Final Mental Checklist

- If code writes through a pointer/reference, identify the actual storage being modified.
- If memory is allocated, identify exact byte count, failure check, owner, and matching free/drop.
- If a string is copied/appended, identify terminator space and destination capacity.
- If a C loop uses indexes, check `<` vs `<=`, length source, and whether the index changes correctly.
- If a C stream is used, check the return value and whether the stream may be buffered.
- If C++ uses base pointers/references, ask: virtual function? virtual destructor? slicing?
- If C++ transfers ownership, look for `std::move` and whether the old owner is left empty but valid.
- If Rust code fails, ask: move, borrow conflict, lifetime, missing trait bound, or semicolon return issue?
- If Rust iterator code is wrong, write the item type at each step before fixing the code.
- If an exam asks 'explain', name the rule, apply it to the code, and state the consequence.

RAG Topic Coverage Checklist

Every topic from the RAG tracker is included below. Red topics should get active recall first, then amber, then green.

C

Status	Topic	Priority bucket
A	Basic C syntax and program layout: `main`, comments, `#include`, blocks, semicolons, `printf`, escape sequences, automatic variables, stack allocation, static typing, format specifiers.	C preprocessor, headers, build
R	Types and conversions: integer division, signed vs unsigned behaviour, promotion, casting, chars, floats, booleans, `typedef`, `size_t`, structs as data types, enums/unions.	C pointers, memory, arrays, strings
A	Arrays and strings: 1D/2D arrays, contiguous memory, indexing, bounds errors, char arrays, null terminator, string formatting/printing, `<string.h>` functions.	C pointers, memory, arrays, strings
A	Pointers and memory: addresses, dereferencing, arrays as pointers, arrays of strings, `malloc`, `free`, dangling pointers, double-free, memory leaks, segmentation faults.	C pointers, memory, arrays, strings
R	Functions with pointers: pass by value vs pointer update, arrays as parameters, pointers to pointers, modifying caller state, allocating inside helper functions.	C pointers, memory, arrays, strings
R	Function pointers and callbacks, including `qsort`.	C pointers, memory, arrays, strings
R	Structures and dynamic data structures: `struct`, `typedef`, member access, linked lists, arrays vs linked lists tradeoffs.	C pointers, memory, arrays, strings
R	Files and streams: `scanf`, standard streams, text vs binary streams, file modes, file I/O, buffering/flushing, `feof`, `ferror`, `fseek`, `rewind`, `errno`, `strerror`.	C files, streams, buffering, diagnostics
R	Stream buffering details beyond `fflush`: implementation-dependent buffer sizes, `setbuf`, and raw unbuffered `read`/`write`.	C files, streams, buffering, diagnostics
R	Preprocessor and macros: `#include`, macro expansion, stringification, search paths, macro pitfalls, precedence problems, safe macro design.	C preprocessor, headers, build
R	Program structure and compilation: separate compilation, headers, APIs, `extern`, `static`, scope/visibility, multiple includes, include guards, simple makefiles and dependencies.	C preprocessor, headers, build
R	Object files and libraries: reusing `.o` files separately from source, linking object files, and libraries as collections of object files.	C preprocessor, headers, build
R	Makefiles and dependency-based builds: targets, prerequisites/dependencies, tab-indented commands, object-file rules, default target, `clean` target, `make` up-to-date behaviour, and rebuilding when headers or source files change.	C preprocessor, headers, build
R	Macro precedence and fully parenthesised macros.	C preprocessor, headers, build
R	Macro vs function tradeoffs.	C files, streams, buffering, diagnostics
R	Macro stringification and predefined macros.	C pointers, memory, arrays, strings
A	Manual string duplication with `malloc`.	C pointers, memory, arrays, strings
A	Allocating 2D arrays with pointers.	C pointers, memory, arrays, strings
R	Global vs `static` variables.	C preprocessor, headers, build
R	Output buffering and forcing `printf` output to appear immediately.	C files, streams, buffering, diagnostics
R	`errno` usage: only inspect it after an actual failure.	C files, streams, buffering, diagnostics
A	Control flow and early problem-solving patterns: `if`, `for`, `while`, factorial, numerical approximation of `e`, compile/link/run workflow.	C preprocessor, headers, build
R	Arrays and memory layout in practice: out-of-bounds access, undefined behaviour, segmentation faults, memory corruption.	C pointers, memory, arrays, strings
G	`sizeof` vs `strlen`, and what each actually measures.	C pointers, memory, arrays, strings
G	Custom string routines: `mystrlen`, `string_copy`, pointer-based string functions.	C pointers, memory, arrays, strings
A	Command-line arguments in C: `argc`, `argv`, quoting, `atoi`, `atof`.	General exam technique and syntax precision
A	Multi-dimensional arrays and array-based simulations, including Game of Life style evolution.	C pointers, memory, arrays, strings
G	Struct-based exercises: points/rectangles, area tests, inside/outside checks.	C pointers, memory, arrays, strings
A	String-processing exercises: substitution/translation, palindrome detection, normalisation, `<ctype.h>` helpers such as `ispunct` and `tolower`.	Rust ownership, borrowing, strings, slices
R	Pointer exercises: basic pointers, pointers with arrays, tricky pointers, pointer arithmetic.	C pointers, memory, arrays, strings
R	Generic low-level programming with `void *`: type-generic swap, comparison functions, generic merge sort.	C pointers, memory, arrays, strings
R	Debugging tools for memory bugs: Valgrind and address sanitiser.	General exam technique and syntax precision
A	Stack implementation practice using linked structures, `push`/`pop`, recursion for printing, and empty-stack edge cases.	C pointers, memory, arrays, strings

C++

Status	Topic	Priority bucket
R	Moving from C to C++: standards, compile/link flow, `iostream`, streams, `bool`, `std::string`, `using`, `auto`, scope resolution, function/operator overloading, default parameters, pass by reference.	C++ templates, operators, STL algorithms
R	C++ build systems at a high level: CMake as the large-project build tool mentioned in the slides, and how it relates to Makefiles/manual dependency management.	C preprocessor, headers, build
A	Classes and structs: declarations, struct vs class, access specifiers, constructors, destructors, parameterised/default/copy constructors, member functions, encapsulation, `this`.	C++ classes, inheritance, polymorphism
A	Memory, pointers, and references: `new`/`delete`, constructors/destructors, `nullptr`, references, pass-by-reference, `const &`, pointers to class members.	C++ classes, inheritance, polymorphism
A	Inheritance: public/protected/private inheritance, base vs derived access, constructor/destructor order, interface design.	C++ classes, inheritance, polymorphism
R	Operator overloading: overloadable vs non-overloadable operators, member vs friend functions, `[]`, `()`, `<<`, least-surprise design.	C++ templates, operators, STL algorithms
R	Templates and STL: macros vs templates, function templates, class templates, type safety, STL containers including `stack` and `vector`, iterators, `auto` with iterators.	C++ templates, operators, STL algorithms
R	STL algorithms and function objects: `binary_search`, sorting, counting algorithms, and passing functions/lambdas/function objects to algorithms.	C++ templates, operators, STL algorithms
A	Error handling and lambdas: `try`/`catch`/`throw`, standard exceptions, good exception practice, lambda syntax, capture by value/reference.	C++ RAII, smart pointers, exceptions
A	Access control details in inheritance: `private`, `protected`, `public`.	C++ classes, inheritance, polymorphism
A	Why derived classes can access `protected` members but not `private` members.	C++ classes, inheritance, polymorphism
G	Getter/setter style access when direct member access is restricted.	C++ classes, inheritance, polymorphism
R	RAII vs manual `malloc`/`free`.	C pointers, memory, arrays, strings
R	Smart-pointer ownership models in C++ vs Rust ownership/reference-counting.	Rust ownership, borrowing, strings, slices
R	C++ shared ownership smart pointers: `std::shared_ptr`, `std::weak_ptr`, reference counts, copyable shared ownership, and when the managed object is destroyed.	C++ RAII, smart pointers, exceptions
G	Splitting classes across header and source files.	C files, streams, buffering, diagnostics
R	Reference parameters in practice, such as swapping values by reference.	Rust ownership, borrowing, strings, slices
R	Constructor/destructor tracing to understand object lifetime.	C++ classes, inheritance, polymorphism
R	Virtual functions and dynamic binding through base-class pointers.	C++ classes, inheritance, polymorphism
R	Member initialiser lists for constructors.	C pointers, memory, arrays, strings
A	Multiple inheritance.	C++ classes, inheritance, polymorphism
R	Smart-pointer practice with `std::unique_ptr`.	C pointers, memory, arrays, strings

Rust

Status	Topic	Priority bucket
A	Tooling and motivation: why Rust, memory-safety goals, `rustc`, `cargo` as build system and package manager, `Cargo.toml`, Rust Book/docs, and `rust-analyzer` editor support.	General exam technique and syntax precision
R	Crash-course basics: `let`, `mut`, shadowing, `const`, integer types, tuples, arrays, runtime bounds checks, printing.	C pointers, memory, arrays, strings
R	Rust functions and control flow: function parameters/return types, unit type `()`, `if` expressions, `loop`/`while`/`for`, loop labels, `break` values, `break`/`continue`, and doc comments `///`.	General exam technique and syntax precision
R	Ownership fundamentals: stack vs heap, `String`, moves, copies, drop, ownership rules, references and borrowing, mutable borrowing, dangling references.	Rust ownership, borrowing, strings, slices
R	Borrowing and slices: references, `&[T]`, `&str`, `String` vs `&str`, avoiding dangling references.	Rust ownership, borrowing, strings, slices
R	Collections and data modelling: `Vec<T>`, strings, structs, methods, associated functions, enums, `Option<T>`, `match`, `if let`.	C pointers, memory, arrays, strings
R	Error handling and file I/O: `panic!`, `Result<T, E>`, explicit error handling, `?`, parsing, file reading, iterating over lines.	Rust Result, Option, parsing, tests
A	Project structure, testing, and `HashMap`: `main.rs` vs `lib.rs`, `pub`, modules, unit tests, integration tests, assertions, `HashMap`, `entry` API.	Rust Result, Option, parsing, tests
R	Generics and traits: generic functions/structs/methods, trait bounds, `where` clauses, default trait methods, derivable traits.	C pointers, memory, arrays, strings
R	Closures, iterators, and lifetimes: closure syntax/capture, `Fn`/`FnMut`/`FnOnce`, iterator pipelines, `map`, `filter`, `enumerate`, `zip`, `take`, `collect`, custom iterators, lifetime annotations, elision rules.	Rust iterators, traits, closures, concurrency
R	Rust move vs copy, and why iterator-based loops avoid C indexing errors.	Rust ownership, borrowing, strings, slices
R	Rust `?` as explicit error propagation without exceptions.	Rust Result, Option, parsing, tests
A	Cargo workflow beyond `build`/`run`: `cargo check`, `cargo fmt`, `cargo clippy`, tests.	Rust Result, Option, parsing, tests
G	Expressions vs statements, and how semicolons affect return values.	General exam technique and syntax precision
R	Command-line input, parsing, and comparing `expect`, `match`, and `?`.	Rust Result, Option, parsing, tests
R	Array/slice processing, threshold-style scans, and tuple destructuring.	C pointers, memory, arrays, strings
R	`Option`, `Some`/`None`, `match`, and `if let` in small data-structure exercises.	Rust Result, Option, parsing, tests
A	Unit tests, integration tests, assertion patterns, and float comparisons with tolerances.	Rust Result, Option, parsing, tests
R	File parsing exercises, including CSV-style parsing into structs and `Box<dyn Error>` return types.	Rust ownership, borrowing, strings, slices
R	Custom iterators and more involved iterator pipelines over `HashMap`/`Vec`.	Rust iterators, traits, closures, concurrency
R	Sorting practical Rust data: `sort_by`, descending order, tie handling, `partial_cmp` for floating-point values, and `Ordering`.	C++ templates, operators, STL algorithms
A	Smart pointers beyond the lecture slides: `Box<T>` and `Rc<T>`.	Rust ownership, borrowing, strings, slices
R	Concurrency topics: spawned threads, `move` closures, joining, `mpsc` channels, `Arc<T>`, `Mutex<T>`, poisoned mutex handling, parallel aggregation.	Rust iterators, traits, closures, concurrency